Class 3: Continuous Deployment

Prerequisites

- Basic understanding of Continuous Integration (CI).
- Familiarity with Jenkins and pipeline creation.
- Knowledge of software deployment processes.

Session Overview

- **Introduction to Continuous Deployment**: Explore the role of CD in the DevOps lifecycle, focusing on extending CI with automated deployment processes in a Node.js environment.
- **CD Pipeline Components**: Discuss the key components of a CD pipeline, including deployment automation, environment provisioning, and testing, with a focus on Node.js applications.
- **Configuring a CD Pipeline**: Step-by-step guide on setting up a CD pipeline using Jenkins with Node.js, including writing Jenkinsfiles for deployment automation.
- **Deployment Strategies**: Review Blue-Green Deployment, Canary Releases, and Rolling Deployments, with practical examples using Node.js applications.

Principles of Continuous Deployment (CD)

Continuous Deployment (CD) is a key practice in DevOps that focuses on the automated deployment of code changes to production as soon as they pass the necessary testing phases. This practice builds upon Continuous Integration (CI) by ensuring that any change that successfully passes all automated tests is automatically deployed to production, eliminating the need for manual intervention. The following sections outline the core principles and importance of CD in the software development lifecycle.

1. Definition and Purpose of CD

Definition:

- Continuous Deployment refers to the practice of automatically deploying every change that passes the automated testing stages directly to the production environment. This means that every update, no matter how minor, is automatically released to users without requiring human intervention.

Purpose:

- The primary purpose of CD is to speed up the software release process by automating the deployment pipeline. By minimizing manual steps, CD reduces the potential for human error, ensures consistency, and allows teams to deliver new features, fixes, and updates to users more quickly and reliably.

2. Automation of the Deployment Process

In traditional software development, deployment often involves numerous manual steps, including code compilation, packaging, transferring files, configuring environments, and restarting services. This manual process can be error-prone and time-consuming. CD automates these steps, ensuring that every deployment is executed in a consistent and repeatable manner.

Key Elements of Automation:

- **Build Automation**: Once code is committed, the build process is automatically triggered. This includes compiling code, running tests, and creating deployable artifacts (e.g., Docker containers, JAR files).
- **Test Automation**: Automated tests (unit, integration, and end-to-end) are executed to ensure that the code is functioning as expected.
- **Deployment Automation**: The deployment process, including environment provisioning and configuration, is automated to ensure that the application is deployed consistently across all environments.

3. Benefits of Continuous Deployment

CD offers several benefits that make it an essential practice in modern software development:

- **Faster Time to Market**: By automating the deployment process, CD enables teams to release new features and updates more quickly. This allows organizations to respond to customer needs and market changes faster.
- **Reduced Risk of Human Error**: Automation reduces the risk of errors that can occur during manual deployment processes, such as misconfigurations or missed steps.
- **Improved Consistency and Reliability**: Since the deployment process is automated, each deployment is consistent and predictable, reducing the chances of environment-related issues.
- **Continuous Feedback**: With CD, feedback loops are shortened, as changes are deployed immediately after they pass tests. This allows developers to receive feedback on their code in near real-time, leading to quicker iterations and improvements.
- **Enhanced Collaboration**: CD fosters collaboration between development and operations teams by creating a shared responsibility for the deployment process. Automation removes bottlenecks and allows teams to work together more effectively.

4. Extending CI with CD

Continuous Deployment builds on the foundation of Continuous Integration by taking the tested code from the CI pipeline and automatically deploying it to production. This extension of CI into CD ensures that the entire software delivery process is automated, from code commit to production release.

- **CI Focuses on Integration**: CI ensures that all code changes are integrated into a shared repository and tested frequently, detecting integration issues early.
- **CD Focuses on Deployment**: CD takes the integrated code from CI and automates its deployment to production. This reduces the time between writing code and delivering it to users, enabling faster and more reliable releases.

5. Continuous Deployment vs. Continuous Delivery

It is important to distinguish between Continuous Deployment and Continuous Delivery, as they are often confused:

- **Continuous Delivery**: In Continuous Delivery, code changes are automatically tested and prepared for release to production. However, the actual deployment to production requires manual approval. This practice ensures that the code is always in a deployable state but allows for human intervention before it is released.
- **Continuous Deployment**: Continuous Deployment takes Continuous Delivery a step further by automating the deployment to production without requiring manual approval. Every change that passes the automated tests is automatically released to users.

6. Challenges and Considerations in CD

While CD offers numerous benefits, it also presents challenges that teams need to address:

- **Testing Reliability**: Since CD relies heavily on automated testing, ensuring the reliability and coverage of tests is crucial. Inadequate testing can lead to bugs being deployed to production.
- **Monitoring and Rollbacks**: With CD, it's essential to have robust monitoring and rollback mechanisms in place. Automated deployments can introduce issues into production, so teams need to quickly detect and resolve problems, including rolling back to a previous stable version if necessary.
- **Cultural Shift**: Implementing CD requires a cultural shift within the organization, with a focus on automation, collaboration, and continuous improvement. This may require changes in processes, tools, and team dynamics.
- **Security Considerations**: Automating deployments also means automating security checks and ensuring that the deployment pipeline is secure. Security must be integrated into every stage of the pipeline, from code scanning to environment configuration.

Hands-On Lab: Setting up a Continuous Deployment (CD) Pipeline Using Jenkins with Node.js

This hands-on lab will guide students through the process of setting up a Continuous Deployment (CD) pipeline using Jenkins for a Node.js application. By the end of this lab, students will have a fully functional CD pipeline that automatically deploys changes to a production environment whenever code is pushed to a GitHub repository.

Step 1: Prerequisites and Setup

Before starting, ensure that the following prerequisites are met:

1. **Install Node.js and npm**:
   - Download and install Node.js from the official website: https://nodejs.org/.
   - Verify the installation by running the following commands in your terminal:

```
node -v
npm -v
```

2. **Install Jenkins**:
   - **Locally**: Follow the installation instructions for Jenkins based on your operating system from the Jenkins official site.
   - **On AWS EC2**: Launch an Amazon Linux 2 instance, SSH into it, and install Jenkins by following the instructions for your OS.
3. **Set Up Git and GitHub**:
   - Install Git on your machine: https://git-scm.com/.
   - Set up a GitHub account and create a new repository for your Node.js project.

Step 2: Create a Simple Node.js Application

1. **Initialize a Node.js Project**:
   - Navigate to the directory where you want to create your project and run:

```
mkdir my-node-app
cd my-node-app
npm init -y
```

- This will create a basic package.json file with default settings.

2. **Create a Simple Express Server**:
   - Install Express.js as a dependency:

```
npm install express --save
```

- Create an index.js file with the following code:

```
const express = require('express');
const app = express();
const port = process.env.PORT || 3000;

app.get('/', (req, res) => {
  res.send('Hello World!');
});

app.listen(port, () => {
  console.log(`App is running on http://localhost:${port}`);
});
```

- Add a start script to package.json:

```
"scripts": {
  "start": "node index.js"
}
```

3. **Test the Application Locally**:
    - Run the application using:

```
npm start
```

    - Visit http://localhost:3000 in your browser, and you should see “Hello World!”.
4. **Push the Project to GitHub**:
    - Initialize a git repository and push your code to GitHub:

```
git init
git add .
git commit -m "Initial commit"
git remote add origin <your-repo-url>
git push -u origin master
```

Step 3: Set Up Jenkins for Continuous Integration

1. **Start Jenkins**:
    - Run Jenkins and open it in your web browser at http://localhost:8080.
    - Complete the initial setup by following the prompts and installing the suggested plugins.
2. **Install Necessary Plugins**:
    - Go to Manage Jenkins -> Manage Plugins.
    - Install the following plugins:
        - **Git Plugin**: For integrating with GitHub.
        - **NodeJS Plugin**: For running Node.js builds.
        - **Pipeline Plugin**: For defining Jenkins pipelines using Jenkinsfiles.
3. **Configure Jenkins to Use Node.js**:
    - Go to Manage Jenkins -> Global Tool Configuration.
    - Scroll down to NodeJS and click Add NodeJS.
    - Name it (e.g., NodeJS 14.x), choose the version, and save.
4. **Create a Jenkins Pipeline Job**:
    - From the Jenkins dashboard, click New Item.
    - Enter a name for the project (e.g., NodeJS-CD-Pipeline) and select Pipeline.
    - Scroll down to the Pipeline section and choose Pipeline script from SCM.
    - Select Git as the SCM and enter your GitHub repository URL.
    - In the Script Path, enter Jenkinsfile.

Step 4: Writing a Jenkinsfile for Continuous Deployment

1. **Create a Jenkinsfile**:
    - In the root of your Node.js project, create a file named Jenkinsfile.
    - Add the following content:

```
pipeline {
  agent any

  tools {
    nodejs "NodeJS 14.x"
  }

  stages {
    stage('Checkout') {
      steps {
        git branch: 'master', url:
'https://github.com/<your-username>/<your-repo>.git'
      }
    }

    stage('Install Dependencies') {
      steps {
        sh 'npm install'
      }
    }

    stage('Run Tests') {
      steps {
        sh 'npm test'
      }
    }

    stage('Build') {
      steps {
        sh 'npm run build'
      }
    }

    stage('Deploy to Production') {
      steps {
        sh '''
        ssh -o StrictHostKeyChecking=no ec2-user@<EC2-Public-IP> <<EOF
        cd /var/www/html
        git pull origin master
        npm install --production
        pm2 restart all
```

```
        EOF
        '''
      }
    }
  }

  post {
    success {
      echo 'Deployment successful!'
    }
    failure {
      echo 'Deployment failed!'
    }
  }
}
```

2. **Explanation of Jenkinsfile**:
    - **Agent any**: This tells Jenkins to run the pipeline on any available agent.
    - **Tools**: Specifies the Node.js version to use.
    - **Stages**: Defines the different stages in the pipeline:
        - **Checkout**: Clones the repository from GitHub.
        - **Install Dependencies**: Runs npm install to install Node.js dependencies.
        - **Run Tests**: Executes tests with npm test.
        - **Build**: Builds the project (if applicable).
        - **Deploy to Production**: Uses SSH to log into an EC2 instance, pull the latest code, install production dependencies, and restart the application using PM2 (a process manager for Node.js).

Step 5: Deploying to an AWS EC2 Instance

1. **Launch an EC2 Instance**:
    - Use the AWS Management Console to launch a new EC2 instance.
    - Select an Amazon Linux 2 AMI and configure it with the necessary security groups (allow SSH and HTTP access).
2. **Install Node.js and PM2 on EC2**:
    - SSH into your EC2 instance:

```
ssh -i /path/to/your-key.pem ec2-user@<EC2-Public-IP>
```

- Install Node.js and npm:

```
curl -sL https://rpm.nodesource.com/setup_14.x | sudo bash -
sudo yum install -y nodejs
```

- Install PM2 globally:

```
sudo npm install -g pm2
```

- Set up your application directory (e.g., /var/www/html) and clone your repository:

```
sudo mkdir -p /var/www/html
cd /var/www/html
git clone https://github.com/<your-username>/<your-repo>.git .
npm install --production
pm2 start index.js
```

3. **Configure Jenkins to Deploy on EC2**:
   - Ensure your Jenkins server can SSH into your EC2 instance using the same SSH key.
   - Update the Jenkinsfile to include the correct EC2 public IP address.
4. **Run the Jenkins Pipeline**:
   - Go to your Jenkins dashboard, select the pipeline job you created, and click Build Now.
   - Watch the pipeline execute each stage, and ensure the application is deployed to your EC2 instance.
5. **Verify the Deployment**:
   - Access the application in your browser using the EC2 public IP address (http://<EC2-Public-IP>). You should see the running Node.js application.

Step 6: Implementing Deployment Strategies

1. **Blue-Green Deployment**:
   - Create two identical environments: Blue (current production) and Green (new version).
   - Update the Jenkinsfile to deploy to the Green environment, test it, and switch traffic over from Blue to Green once the deployment is successful.
2. **Canary Releases**:
   - Deploy the new version to a small subset of users or servers (Canary).
   - Monitor the performance and gradually increase the deployment if no issues arise.
3. **Rolling Deployments**:
   - Deploy the new version to a few servers at a time while keeping the rest on the old version.
   - Continue rolling out the deployment until all servers are updated.

Deployment Strategies in Continuous Deployment (CD)

Deployment strategies are crucial in ensuring that new code changes are delivered to users in a reliable, efficient, and low-risk manner. Different strategies offer various benefits depending on the use case, the application architecture, and the business requirements. Below are some common

deployment strategies:

1. Blue-Green Deployment
Overview:
Blue-Green Deployment is a strategy where two identical environments, named Blue and Green, are maintained. At any time, one environment serves live production traffic (e.g., Blue), while the other (e.g., Green) is idle and used for testing new deployments.

Process:
- **Deploy**: The new version of the application is deployed to the idle environment (e.g., Green).
- **Test**: Thorough testing is performed in the Green environment to ensure everything works as expected.
- **Switch Traffic**: Once testing is successful, traffic is switched from the Blue environment to the Green environment, making Green the new live environment.
- **Roll Back**: If any issues arise, traffic can quickly be switched back to the Blue environment, minimizing downtime and user impact.

Advantages:
- **Minimal Downtime**: Traffic switch is nearly instantaneous, reducing downtime during deployment.
- **Easy Rollback**: In case of failure, reverting to the previous version is quick and straightforward.
- **Testing in Production-like Environment**: Testing occurs in an environment identical to production, reducing the risk of environment-specific issues.

Disadvantages:
- **Infrastructure Cost**: Requires maintaining two identical environments, which can be costly.

2. Canary Deployment
Overview:
Canary Deployment is a strategy where a new version of an application is released to a small subset of users or servers before gradually rolling it out to the entire user base. This allows for careful monitoring and risk mitigation.

Process:
- **Deploy to a Small Group**: Deploy the new version to a small percentage of users or a limited number of servers.
- **Monitor**: Monitor the performance and behavior of the new version, focusing on metrics like error rates, latency, and user feedback.
- **Gradual Rollout**: If no major issues are detected, incrementally increase the percentage of users or servers receiving the new version until the entire user base is migrated.
- **Rollback**: If problems are identified, the deployment can be halted, and the previous version can be restored for the affected users.

Advantages:
- **Reduced Risk**: By initially deploying to a small group, potential issues are limited in scope,

reducing the risk of widespread impact.

- **Continuous Feedback**: Allows for real-time feedback and monitoring, providing the opportunity to make adjustments before a full rollout.
- **User Segmentation**: Enables testing of new features with specific user groups, providing valuable insights.

Disadvantages:

- **Complexity**: Requires careful planning and monitoring, making it more complex to implement than other strategies.
- **Gradual Rollout Time**: The process can be slow, especially for large user bases.

3. Rolling Deployment

Overview:

Rolling Deployment is a strategy where the new version of an application is deployed incrementally to a subset of servers or instances. Each server is updated one at a time or in small batches, allowing the old version to remain active on the remaining servers until the entire deployment is complete.

Process:

- **Batch Deployment**: Divide servers into batches and deploy the new version to one batch at a time.
- **Monitor**: After each batch is updated, monitor the performance and health of the application.
- **Continue or Rollback**: If no issues are detected, proceed to the next batch. If problems occur, rollback the changes for the affected batch.
- **Complete Rollout**: Continue the process until all servers have been updated.

Advantages:

- **No Downtime**: Since the application is gradually rolled out, it continues to run on the old version while the new version is being deployed, minimizing or eliminating downtime.
- **Resource Efficiency**: Does not require duplicate environments like Blue-Green Deployment, making it more resource-efficient.
- **Scalability**: Suitable for large-scale applications with many servers or instances.

Disadvantages:

- **Partial Rollback Complexity**: Rolling back during a partial rollout can be complex, as some servers may already be running the new version while others are not.
- **Deployment Speed**: Slower compared to a full deployment, especially for large-scale systems.

4. A/B Testing Deployment

Overview:

A/B Testing Deployment is a strategy where different versions of an application are deployed simultaneously to different segments of users. This is primarily used to test and compare the performance of two or more versions to determine which one performs better in terms of user engagement, conversion rates, etc.

Process:

- **Deploy Multiple Versions**: Deploy version A to one user group and version B to another.
- **Analyze User Behavior**: Collect data on user interactions, performance metrics, and other relevant factors.
- **Select the Best Version**: Based on the analysis, determine which version performs better and deploy it to the entire user base.

Advantages:

- **Data-Driven Decisions**: Provides concrete data to inform decisions on which version to roll out.
- **User Feedback**: Allows for real-time user feedback and testing of new features or changes.

Disadvantages:

- **Complexity**: Requires infrastructure to support multiple versions and the ability to segment users effectively.
- **User Experience Consistency**: Different user groups may have different experiences, leading to inconsistencies.

5. Shadow Deployment

Overview:

Shadow Deployment is a strategy where the new version of an application is deployed alongside the current version but without serving live traffic. It receives a copy of the live traffic to test the new version's performance and stability without impacting actual users.

Process:

- **Deploy Shadow Version**: Deploy the new version in a shadow environment where it receives real traffic data, but the responses are not sent to users.
- **Monitor and Compare**: Compare the behavior and performance of the shadow deployment against the live version.
- **Deploy Live**: If the shadow deployment performs well, it can be promoted to production.

Advantages:

- **Risk-Free Testing**: Allows for real-world testing without impacting users, minimizing risk.
- **Performance Benchmarking**: Provides a way to benchmark the new version against the existing version using live data.

Disadvantages:

- **Infrastructure Overhead**: Requires additional infrastructure to support the shadow deployment, leading to higher costs.
- **No User Feedback**: Since the shadow deployment does not interact with users, it does not provide direct user feedback.